

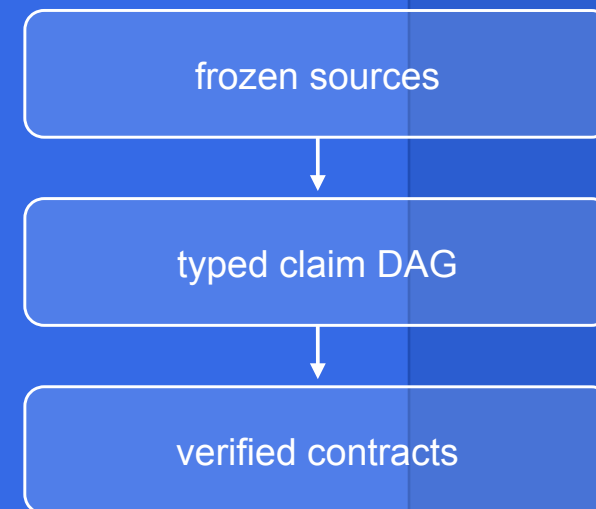
VERIFICATION-AWARE INCREMENTAL SEARCH

AgtXlv

From a theorem query to a reusable dependency path

Math-first pilot specification

Running example: graph-theoretic nonstabilizerness / v0.4 / 2026



Search returns a reading list. You needed a build receipt.

A question a theorist actually asks: **why is $\text{RoM}_m(\rho) = \max(1, \max_q \sum |\text{tr}(P\rho)|)$ exact when G_m is a perfect graph?**

WHAT A SEARCH ENGINE RETURNS

- a ranked list of related papers
- citation counts and abstracts
- no statement of what is actually used

WHAT AGTXIV RETURNS

- the claim's address in a contract registry
- its load-bearing imports, with versions
- the blocked frontier and the local delta left to build

Four questions that must not be collapsed

1

Source fidelity

What did the paper actually state?

2

Mathematical correctness

Does the conclusion follow from the encoded assumptions?

3

Computational reproducibility

Can the reported number be regenerated?

4

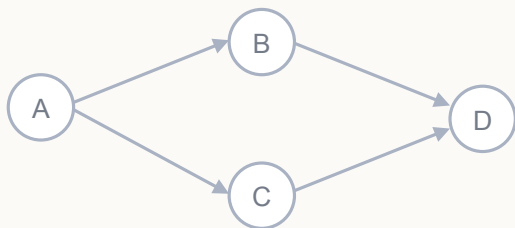
Physical-semantic alignment

Does the formal object mean the intended physics?

A clean Lean build answers only question 2 — and only under the definitions you encoded.

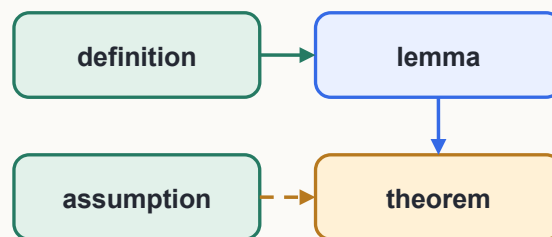
A citation graph is not a proof graph

Citation graph



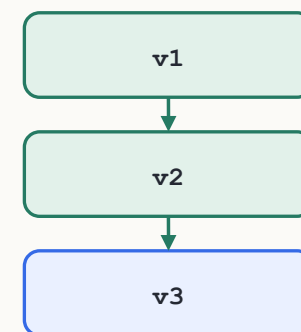
A cites B — for what reason?

Typed claim DAG



which premises are load-bearing?

Lean support graph



which declarations were checked?

A cites B $\Rightarrow$ A uses a theorem of B, or that B's hypotheses hold in A.

The reusable unit is a claim, not a paper

```
math-contract:claim.closed-form-rom
```

normalized statement	quantifiers, domain, exactness
assumptions	no Pauli-active dependency; G_m perfect
theorem imports	exact-graph-program, perfect-graph-antiblocker
source anchor	draft.tex span, frozen bundle hash
Lean binding	declaration or an explicit GAP
status vector + blockers	per-axis, never one Boolean

Why claim-level

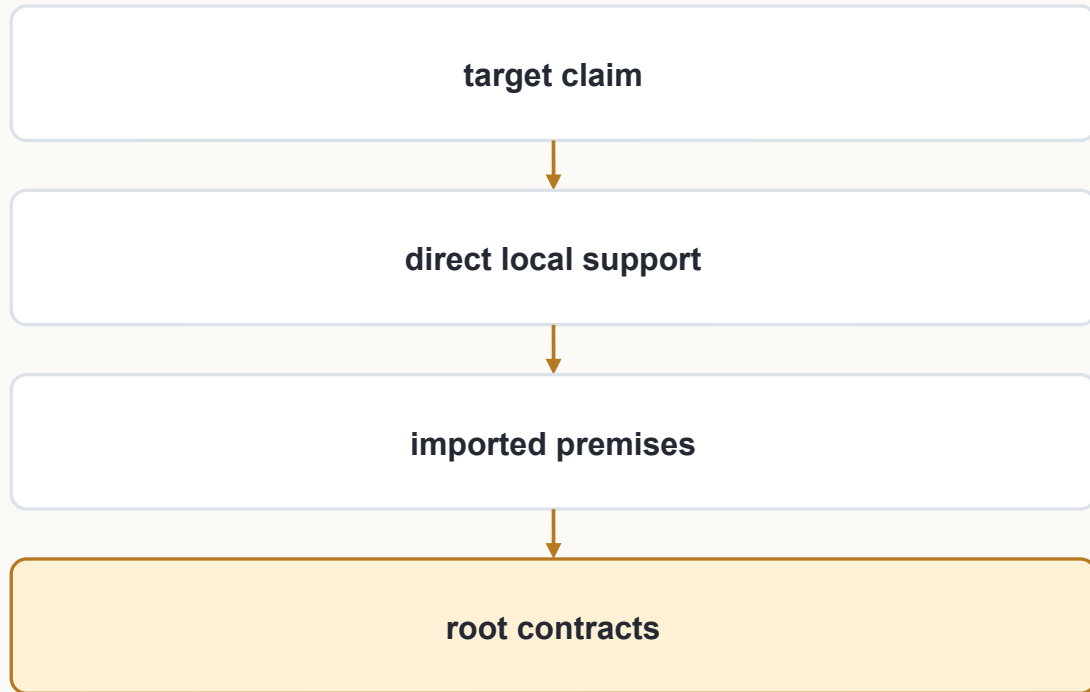
A paper exports many results at different maturity. Reuse binds to one statement, one version.

Progressive standardization

INDEXED
NORMALIZED
DEPENDENCY_MAPPED
FORMALLY_CONNECTED
ACCEPTED_CONTRACT

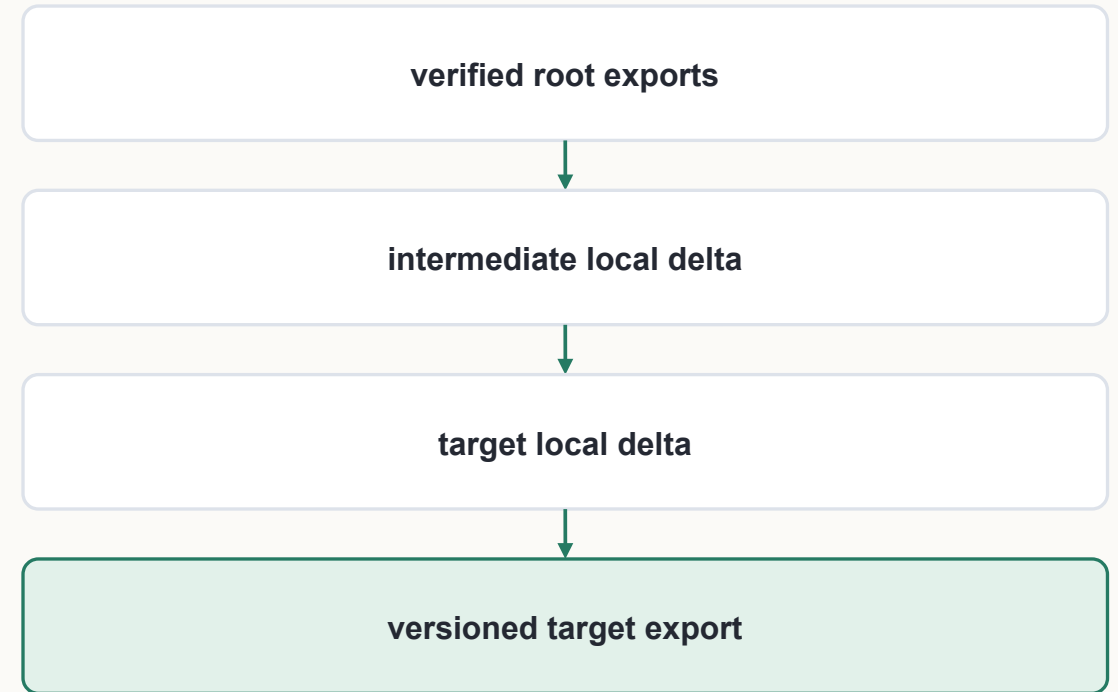
Trace backward, then verify forward

BACKWARD — dependency archaeology



Stop at accepted contracts, reusable packages, or a declared frontier.

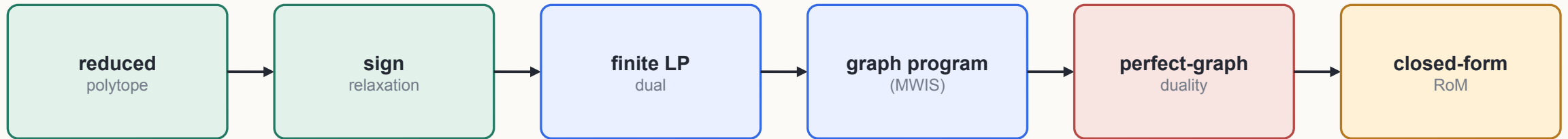
FORWARD — verification build



Prove only the residual delta; reuse every accepted contract.

WORKED EXAMPLE

The running example, as the registry stores it



● Lean-linked

● local derivation

● unresolved external

● target delta

TARGET CONTRACT

`claim:closed-form-rom`

`LOCAL_DERIVATION_WITH_BLOCKED_IMPORTS`

WHAT BLOCKS IT

- root contract not yet accepted
- weighted perfect-graph duality not frozen from the primary source

Where the frontier actually is

74

DAG nodes

129

typed edges

10

Lean-linked

45

declared GAP

8

unresolved external

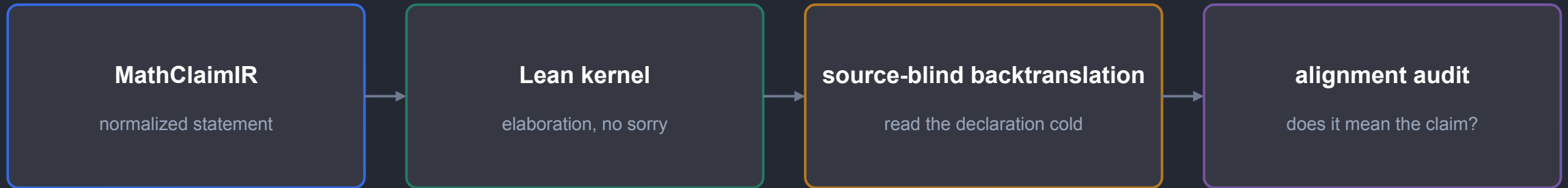
8

open blockers

The registry reports GraphComplete = FALSE.

45 of 74 nodes carry an explicit GAP and 8 external mathematical interfaces remain unresolved. Those are recorded states, not omissions — nothing is promoted to accepted without source alignment and a verification record.

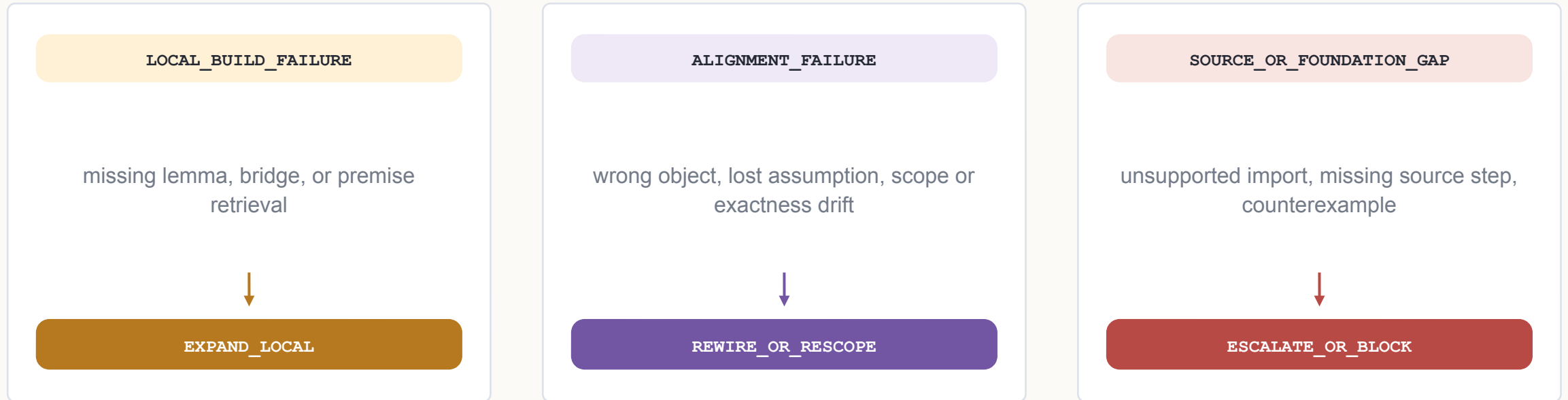
Lean is the kernel, not the oracle



KERNEL_CHECKED means the conclusion follows from the definitions you encoded.

It does not certify that the encoding captures the intended physical object, the approximation regime, or the observable. That is a separate axis with a separate reviewer.

A failed check refines the graph locally



Only the affected region is rebuilt. The rest of the accepted closure is reused unchanged.

Two different reds, found by the audit

PROOF_GAP

THEOREM_NOT_REFUTED

V-representation import

The root reconstruction is still pending, so the import cannot be accepted yet. The registry records a repair route and keeps the downstream claim conditional.

The statement is not claimed to be false.

CLAIM_FALSE

EXPLICIT_COUNTEREXAMPLE

Fixed-window monotonicity

For $M = \{X, Z\}$, a free $\pi/2$ Clifford rotation about X raises the reduced functional of a suitable pure qubit from 1 to $\sqrt{2}$.

So it must not be exported as a resource monotone.

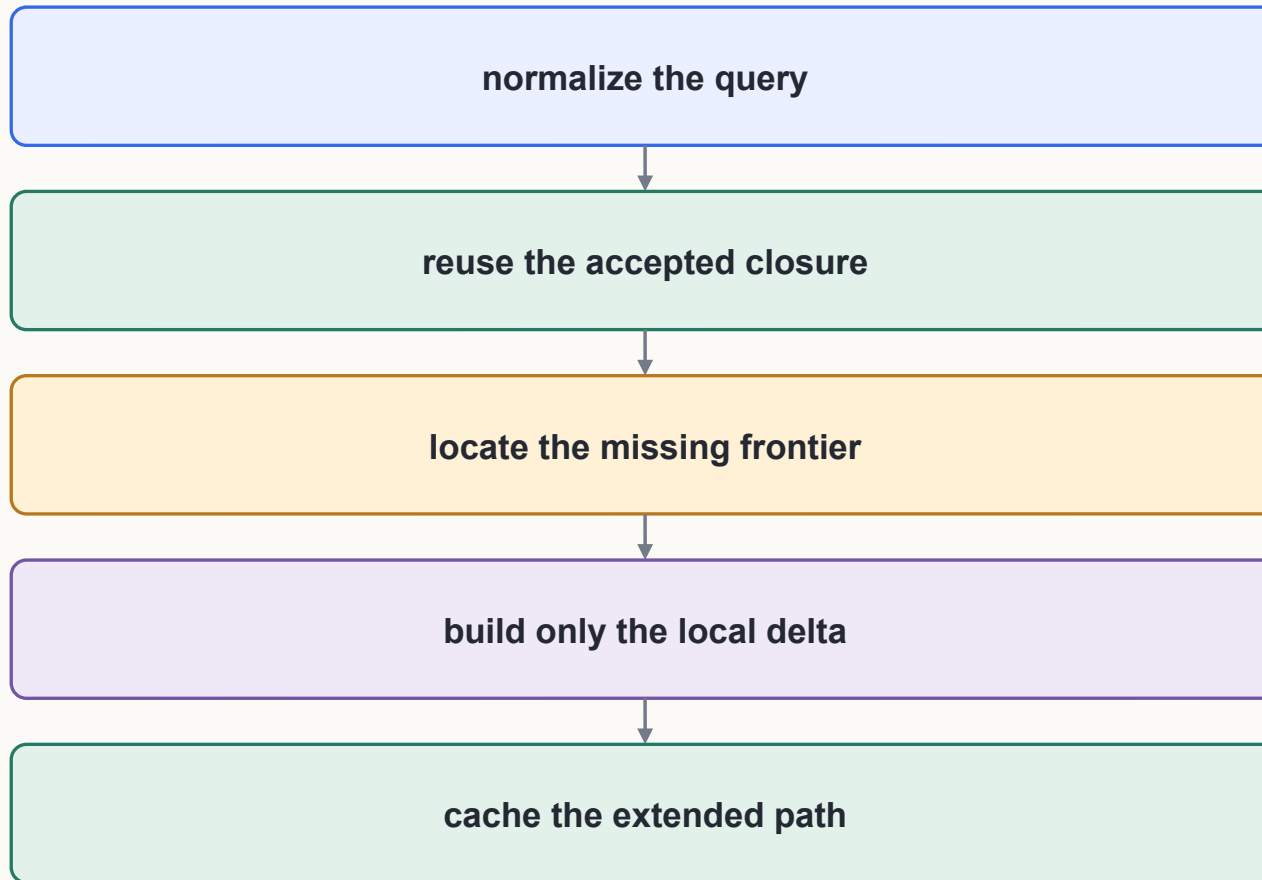
A registry that merges these two into one “red” has destroyed the information a reader needs.

A claim carries a status vector, not one Boolean

source fidelity	PASSED	dependency closure	COMPLETE
mathematics	KERNEL_CHECKED	formal alignment	AUTO_ALIGNMENT_PASSED
physical semantics	AGENT_REVIEWED	approximation regime	PARTIALLY_CHECKED
empirical support	PARTIAL	computation	NOT_APPLICABLE

VERIFICATION_CLOSED means every axis relevant to the declared scope is acceptable — for this query. It is not universal certainty, and it never propagates upward on its own.

Search engine + package manager + incremental build



Reuse is the point

A later query that shares a prefix does not rebuild it. It inherits the accepted closure and pays only for its own delta.

Breaking changes

An upstream contract that changes version invalidates only the affected downstream closure — not the registry.

Search proposes. Only verification promotes.

What a first release has to prove

MINIMUM CONTENT

- 1 target Paper Agent
- 1–3 Root Agents
- 5–15 accepted statements
- 1–3 Lean declarations

EVIDENCE OF THE METHOD

- source-blind backtranslation
- formal-alignment audit
- one real repair record
- adversarial review + clean rebuild

The milestone is not broad coverage. It is one complete, inspectable path from a target claim to its roots and back.

TAKEAWAY

**Prefer an explicit, incomplete registry
with a visible frontier**
to a fluent, unverifiable account of the literature.

claim-level contracts

not whole papers

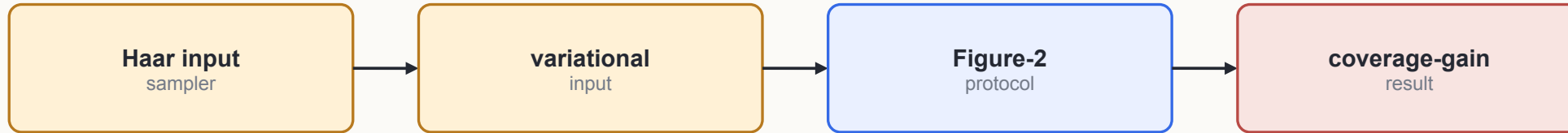
four separate axes

never one Boolean

the frontier is reported

not hidden

Backup — the same machinery on the computation axis



Lean binding: NOT_APPLICABLE

NOT_REPRODUCED

The source bundle ships the rendered figure — but no code, raw samples, seeds, sampling law, angle distribution, precision, tolerance, or error bars.

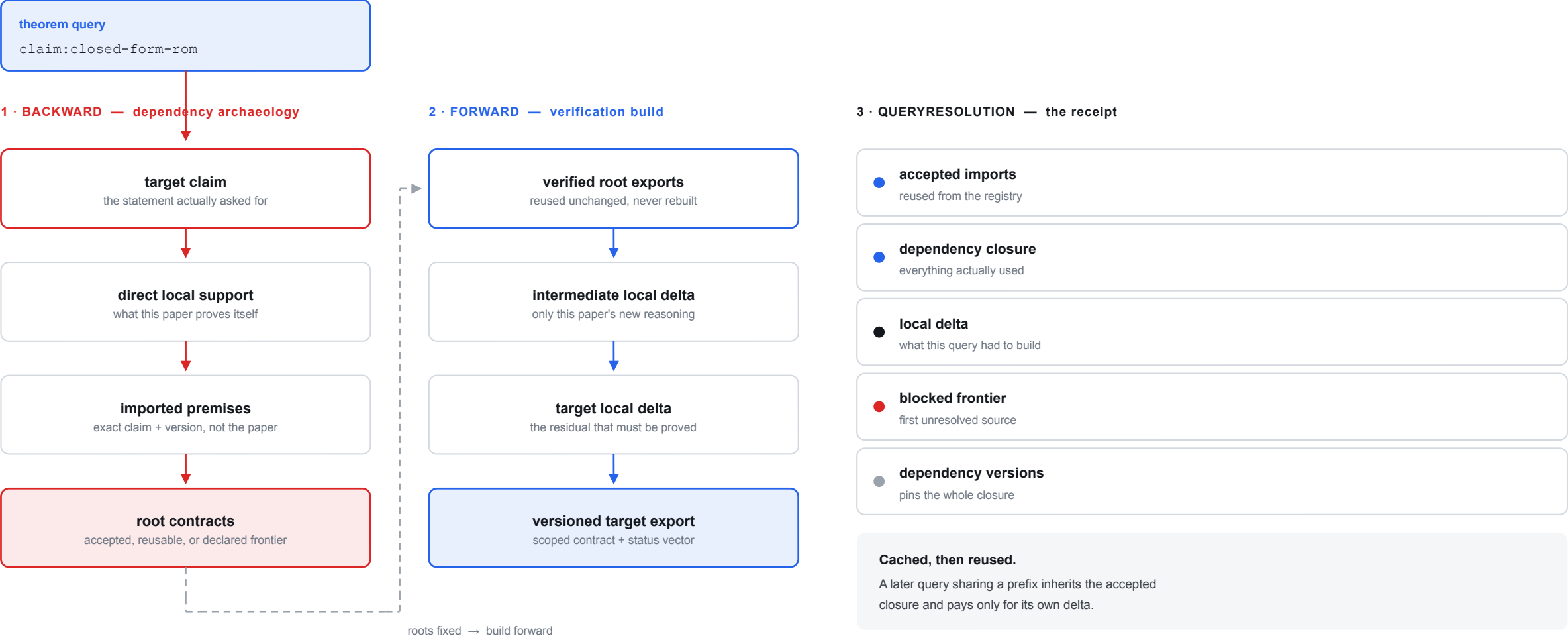
Why this is the useful contrast

Same registry, same status machinery — but the blocker is code and metadata, not mathematics. Merging it into the math axis would misreport both.

Example A blocked on mathematics. Example B blocked on computation. **That is exactly why the four questions stay separate.**

Resolving a theorem query into a verified dependency path

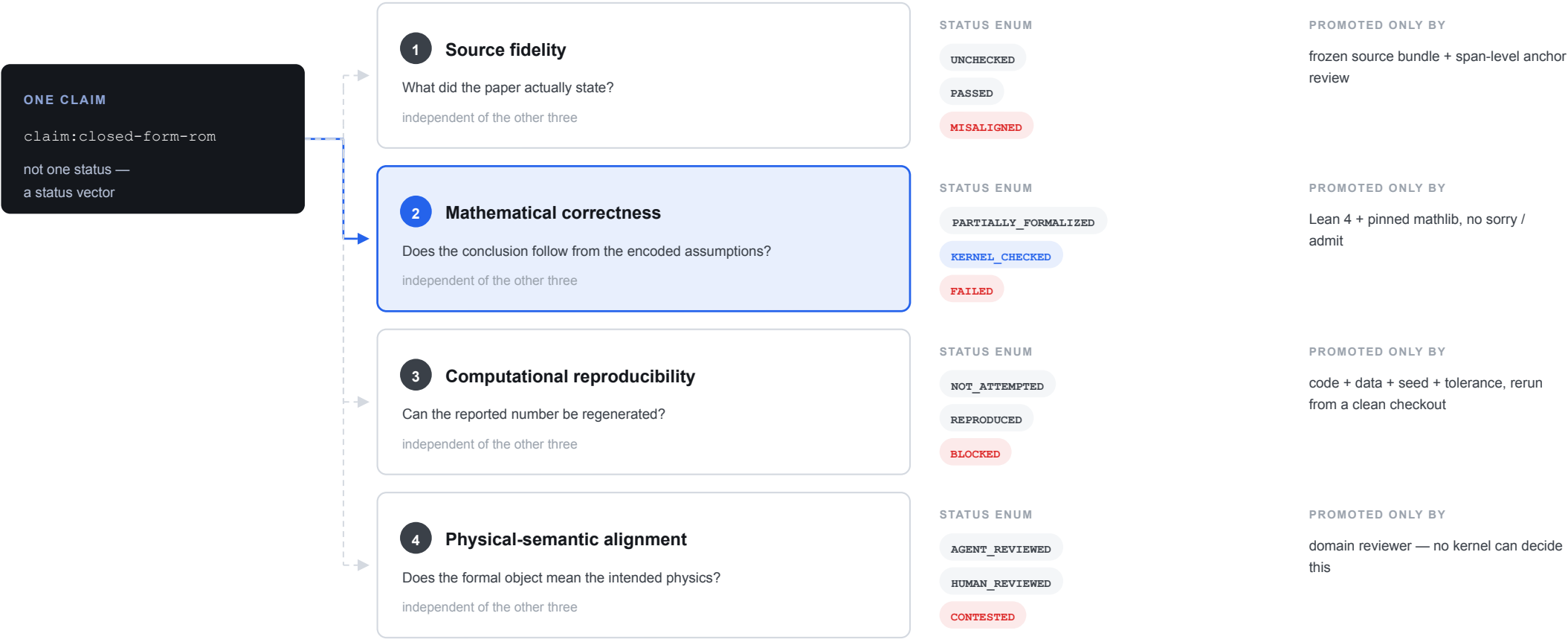
A query is answered by a build receipt, not a reading list. Backward archaeology finds the load-bearing imports; forward verification rebuilds only what is missing.



Two invariants: the reusable unit is the claim-level contract, not the paper — and the frontier is reported, never hidden.

One claim carries four independent verification axes

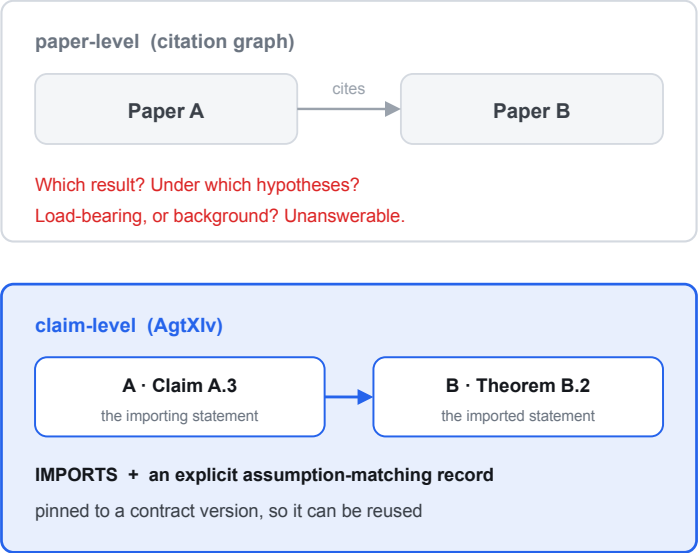
Each axis answers a different question, has its own status enum, and is promoted by its own evidence. Collapsing them into a single Boolean destroys the information a reader needs.



The reusable unit is a claim-level contract, not a paper

A citation binds two documents. A MathContract binds one normalized statement to one version, so a later query can import it without re-reading the paper.

WHAT THE DEPENDENCY BINDS TO



WHY THIS MATTERS

A paper exports many results at different maturity.
Binding reuse to the paper inherits all of them —
including the ones that are still unverified.

MATHCONTRACT — WHAT ONE RECORD MUST CARRY

math-contract:claim.closed-form-rom v0.4

● normalized statement	quantifiers · domain · exactness
● assumptions	no Pauli-active dependency; G_M perfect
● theorem imports	exact-graph-program · perfect-graph-antiblocker
● source anchor	frozen bundle hash + span in draft.tex
● Lean binding	declaration, or an explicit GAP — never blank
● status vector	one value per axis, never one Boolean
● blockers	what stops promotion, named and open

An importer binds to contract@v0.4 — not to “the paper”.

An upstream version bump invalidates only the affected downstream closure.

A CONTRACT NEED NOT REACH ACCEPTANCE IN ONE PASS



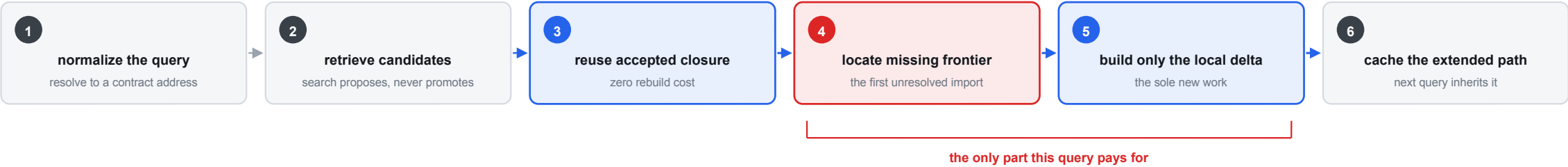
The registry behaves like a package manager with an incremental build

The expensive offline path standardizes contracts once. The online path reuses the accepted closure and pays only for the frontier that is actually missing.

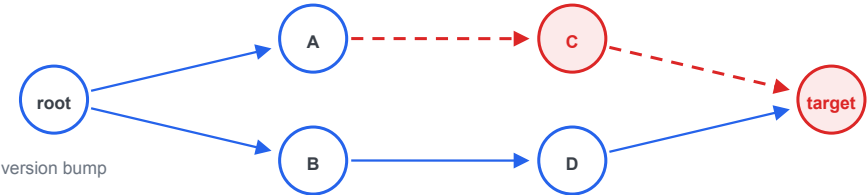
OFFLINE — done once, amortised over every later query



ONLINE — what a single query actually costs



WHEN AN UPSTREAM CONTRACT CHANGES



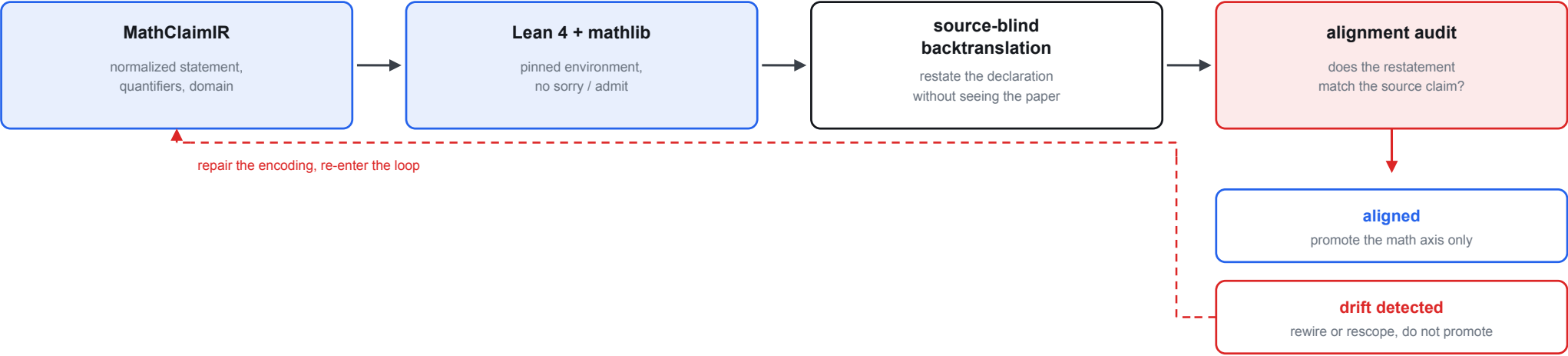
● invalidated — must be rechecked

● untouched — stays accepted, no rebuild

Invalidation follows the dependency edges, not the calendar.

Lean is the trusted kernel — it is not an oracle for physics

The kernel decides one thing: does the conclusion follow from the encoded assumptions. Whether the encoding is the intended object is decided by a separate, source-blind audit.



KERNEL_CHECKED

The conclusion follows from the definitions and assumptions you encoded.

Mechanically checkable. Reproducible from a clean environment.

STILL UNDECIDED

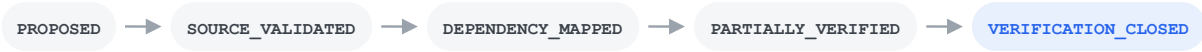
Whether that encoding is the physical object, regime, and observable you meant.

Needs a domain reviewer. No kernel can settle it.

A failed check refines the graph locally — it never silently promotes

Every node carries a lifecycle state. A failure is classified before it is repaired, and two different kinds of red are never merged into one.

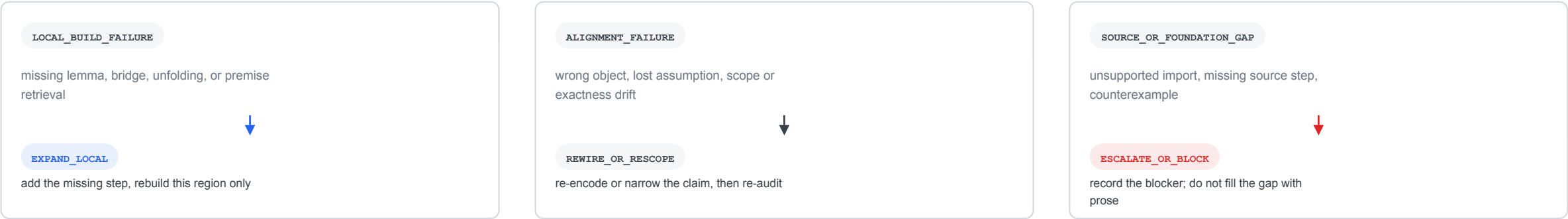
LIFECYCLE OF A NODE, CHAIN, OR EXPORT



VERIFICATION_CLOSED is query-scoped: every axis relevant to the declared scope is acceptable — for this query.

It is not universal certainty, and it does not propagate upward on its own.

A FAILURE IS CLASSIFIED BEFORE IT IS REPAIRED



TWO DIFFERENT REDS — MERGING THEM DESTROYS THE INFORMATION

